

High-Fidelity Quranic Verification: Serverless & Expo Architecture

1. The Core Vision: Speed and Simplicity

This document outlines the complete engineering blueprint for a real-time Quranic recitation verification system. Our goal is to detect mistakes in pronunciation, timing, and Tajweed (rules of recitation) instantly.

We are rejecting the traditional "heavy" server approach (Kubernetes) in favor of a **Serverless** model. This allows us to pay only for what we use while scaling infinitely. On the mobile side, we are pushing the limits of **Expo** by injecting high-performance C++ code directly into the app to handle audio with zero lag.

2. Mobile Architecture: Expo + Native Speed

The mobile app is not just a UI; it is a high-speed sensor. Standard React Native is too slow for real-time audio analysis because it constantly converts data into text (JSON) to send it between the "Native" side (microphone) and the "JavaScript" side (app logic).

We will fix this using **JSI (JavaScript Interface)**. This allows our JavaScript code to "hold hands" with the C++ memory directly, skipping the slow conversion process.

2.1 The "Zero-Copy" Trick

In a typical app, when the microphone captures sound, that data is copied to send it to the JavaScript layer. Copying takes time and battery.

- **Our Approach:** We create a **Shared Object**. This is a block of memory that both C++ and JavaScript can read.
- **How it works:** The microphone writes audio data into this block. The JavaScript UI reads from the exact same block to draw waveforms. No copying happens. This is the only way to achieve our sub-10ms latency goal.

2.2 The Audio Engine

We need to bypass the phone's standard audio processing to get the raw sound faster.

- **Android (Oboe):** We will use Google's **Oboe** library. It automatically selects the fastest audio path (AAudio or OpenSL) for the specific Android device. It also uses "Exclusive Mode" to bypass the Android mixer, cutting latency significantly.
- **iOS (Core Audio):** We will wrap Apple's standard **Audio Units** in C++ so our code looks the same for both Android and iOS.

3. Cloud Architecture: The "Invisible" Server

We are replacing complex server clusters with two specialized services: **Google Cloud Run** for connections and **Modal** for intelligence.

3.1 The Traffic Controller: Google Cloud Run

We need a server that keeps a connection open with the user's phone to send feedback (like "You missed a Madd").

- **The WebSocket Gateway:** We will deploy a lightweight Go or Node.js server on **Google Cloud Run**. Its only job is to hold the phone line open (WebSocket).
- **Why Cloud Run?** It scales automatically. If 10,000 users open the app for prayer, Google adds more servers instantly.

3.2 The Brain: Modal (Serverless GPU)

When the user finishes a verse, we need a super-computer to check the pronunciation.

- **Modal:** This service gives us access to massive GPUs (like the NVIDIA A10G) for just a few seconds at a time.
- **The Flow:**
 1. Cloud Run buffers the audio.
 2. It sends the audio to Modal.
 3. Modal wakes up, runs the AI model, returns the score, and goes back to sleep.
- **Cost:** We pay strictly for the seconds the GPU is running. \$0 cost when no one is reciting.

4. Implementation Roadmap: Small, Precise Tasks

We have broken the build down into small, solvable engineering problems.

Phase 1: The Native Foundation (Weeks 1-4)

Goal: Get C++ code running inside an Expo app.

- **Task 1.1: Expo CNG Setup**
 - **Action:** Initialize a new app with `npx create-expo-app`.
 - **Config:** In `app.json`, set `"newArchEnabled": true`. This turns on the TurboModule system we need for high speed.
 - **Native Config:** Install `expo-build-properties` to lock the Android NDK version (e.g., `"25.1.8937393"`) to ensure our C++ compiles correctly.
- **Task 1.2: Create the Audio Module**
 - **Action:** Run `npx create-expo-module audio-core --local`. This creates a modules folder in your project.
 - **Config:** Open `modules/audio-core/expo-module.config.json` and add `"cpp": true`.

This tells Expo to look for C++ code.

- **Task 1.3: Link Oboe (Android)**

- **The Challenge:** Android makes linking C++ libraries hard.
- **The Fix:** In your module's android/build.gradle, enable prefab. This lets us pull Oboe as a pre-built binary.
- **CMake:** Create a CMakeLists.txt file in android/src/main/cpp. Add find_package(oboe REQUIRED CONFIG) to link the library automatically.

Phase 2: The "Zero-Copy" Pipeline (Weeks 5-8)

Goal: Pass audio data from C++ to JS without crashing.

- **Task 2.1: The Ring Buffer**

- **Concept:** Audio comes in fast. JS processes it slowly. If we don't buffer, we lose data.
- **Action:** Write a C++ class RingBuffer. It needs read() and write() methods.
- **Safety:** Use std::atomic for the read/write index pointers. This prevents the "Audio Thread" and "JS Thread" from fighting over memory (Race Conditions).

- **Task 2.2: Shared Object Implementation**

- **Action:** In your C++ module, inherit from expo::SharedObject.
- **Code:** Implement a function getBuffer() that returns a jsi::ArrayBuffer.
- **Result:** In JavaScript, you can now write const buffer = audioModule.getBuffer(). This variable is now a direct link to the C++ memory.

Phase 3: The Cloud Brain (Weeks 9-12)

Goal: Connect the app to the cloud.

- **Task 3.1: Modal Inference Function**

- **Action:** Write a Python script verifier.py using the modal library.
- **Optimization:** Use the @app.image decorator to download the Wav2Vec2 model *during the build*.
- **Why:** If we download the model when the user requests it, they will wait 20 seconds. If we bake it into the image, they wait 0.5 seconds.

- **Task 3.2: The WebSocket Gateway**

- **Action:** Write a simple Go server.
- **Logic:** When it receives binary audio, append it to a buffer. When it receives a "Pause" flag, send that buffer to the Modal API via HTTP.
- **Deploy:** Deploy to Cloud Run with min-instances=1. This costs ~\$20/month but ensures the first user doesn't wait for the server to wake up.

Phase 4: Intelligence & Feedback (Weeks 13-16)

Goal: Make it smart.

- **Task 4.1: On-Device Voice Detection (VAD)**

- **Action:** Install onnxruntime-react-native.

- **Model:** Download the silero_vad.onnx model (it's tiny, ~2MB).
- **Logic:** Feed audio chunks into this model. If it returns < 0.5 (silence), throw the audio away. Do not send it to the cloud. This saves massive bandwidth costs.
- **Task 4.2: Visual Feedback**
 - **Action:** Use react-native-skia for the waveform.
 - **Optimization:** Do not use React State (useState) to update the waveform. It is too slow. Use a SharedValue from Reanimated to drive the animation directly from the audio data.

5. Why This Wins

- **Zero DevOps:** We aren't managing Kubernetes clusters. We define our infrastructure in code (Modal & Cloud Run).
- **Performance:** By using JSI and C++, we remove the "Bridge Tax" that makes most React Native audio apps feel sluggish.
- **Scalability:** If a million users join for Ramadan, Modal automatically spins up thousands of GPUs. When they leave, it spins down to zero.